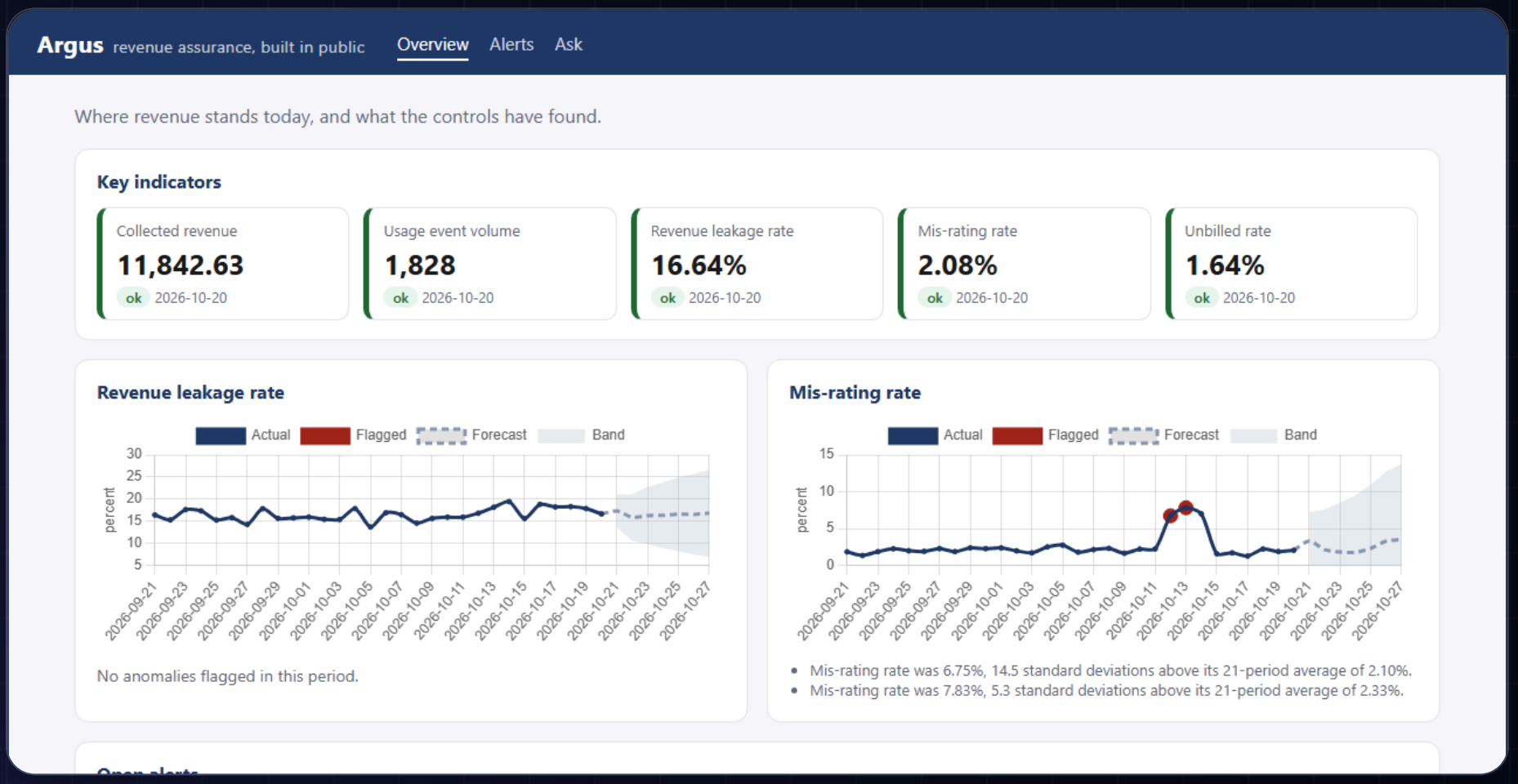
// ARGUS · OPEN SOURCE

20 years chasing money that should have been collected.

So I built the system I wished I'd had.

An AI revenue-assurance app. The AI explains what happened. It never calculates a number.



// THE PROBLEM

Revenue doesn't disappear loudly. It leaks.

A billing feed drops a day. A tariff is applied wrong to one customer class. One site stops sending half its records.

Each is small. None raises an error. And averages hide them:

1 of 4

planted faults caught by network-wide KPIs

One site losing 85% of its volume barely moves the total. Rules that judge every site separately caught all four.

// HOW IT'S BUILT

Six services, four datastores, one command.

deck
Dashboard. Holds no credentials.

pulse
KPIs, anomalies, forecast.

sentry
Rules per site. Alerts close themselves.

analyst
The AI agent. Read-only tools.

prism
The only place SQL is written.

codex
Policy documents, hybrid search.

`ClickHouse` `PostgreSQL` `pgvector` `Valkey`

Every service is configured in YAML and refuses to start on a bad config, reporting every problem at once.

// RULE 1

The AI never writes SQL. It fills in a form.

Every question goes through one query layer, with a closed list of fields. A name that isn’t on the list can’t be selected, filtered or sorted.

```
model asks for revenue_lost → rejected
model asks for site; DROP TABLE → rejected
model asks for leaked, by site → query written by the platform
```

TESTED FIRST 14 attack-shaped field names, all blocked

BONUS The dashboard and the chat can’t disagree

// RULE 2

Numbers and policies never share a tool.

DATA TOOLS

“How much leaked at Ridge?”

An exact figure, computed by a query. Do not use for policy questions.

KNOWLEDGE TOOL

“Who approves a 7,000 write-off?”

A quote from the policy, with its section. Do not use for live figures.

A model that isn’t told the difference answers a policy question from a stale number.

// RULE 3

SQL makes every number. A check proves the AI quoted it right.

Every figure in an answer is looked up in the data the tools returned. The screen says whether all of them were found.

WHAT IT TAUGHT ME

“Accuracy fell to 52.8%”

52.8% was the error rate. The model read our label the way most people would. Now every value says what it measures.

DENIED

Run rules, edit alerts, write SQL, change config, browse the web

// THE EVIDENCE

A demo that says “it works” proves nothing.

Planted faults caught	4 / 4
Fake “fault” (a holiday) ignored	yes
Keyword search	13 / 16
Vector search	12 / 16
Hybrid search	**15 / 16**
AI analyst, golden questions incl. 3 traps	10 / 10
Unit tests, no network, no key	177

The acceptance gate fails if zero cases ran. “No alerts” and “broken detector” look identical otherwise.

// WHAT BUILDING IT TAUGHT ME

→ Averages hide the fault you’re looking for.
Judge every site separately.

→ An anomaly can teach the detector it’s normal.
Fixed rules sit under the statistics.

→ Unit tests don’t replace the real engine.
114 passed while two queries failed for real.

→ Guard rails need tests too.
The number check once flagged the user’s own question.

Open source, MIT. Runs on a laptop.

The code, the evidence and a lesson for each idea: link in the comments.